SAEMPHYSICS

Future Roadmap

Knowledge-to-Lecture Engine for Mathematics, Physics & Computer Science

Source-grounded knowledge extraction • Mathematical verification • Original visual storytelling • Repeatable video production

1. North star

Mission

Build a source-grounded knowledge-to-lecture system that transforms verified mathematics, physics and computer science knowledge into original visual lessons. The system should assist the teacher, not replace scientific judgment.

Core rule

Never go directly from an unverified source or a raw AI response to a published video. Every claim needs provenance, every important equation needs checks, every scene needs a learning purpose, and every episode needs human approval.

2. Target architecture

Source layer

Permitted MIT OpenCourseWare materials, open textbooks, public-domain historical works, trusted reference books, research papers where appropriate, and teacher-created notes.

Ingestion layer

Discover resources; record metadata and license; extract HTML/PDF/text/captions/equations; preserve page or section locations; normalize into structured records; detect duplicates; flag extraction failures.

Knowledge layer

Store concepts, definitions, assumptions, equations, proofs, examples, prerequisites, misconceptions, source passages, and review status. Use full-text or vector retrieval for source passages and a concept graph for relationships.

Reasoning layer

Use language models for drafting and critique; SymPy for symbolic checks; NumPy and numerical solvers for simulation; dimensional and limiting-case tests; trusted references and teacher review for final validation.

Pedagogy layer

Create audience-specific learning objectives, conceptual questions, narrative beats, derivations, demonstrations, worked examples and assessments.

Lecture compiler

Convert an approved lesson plan into a structured scene specification, then into Manim or other renderable code. Keep the intermediate specification independent of any one renderer.

Production layer

Render previews, record or generate permitted narration, synchronize audio, create and correct captions, edit, export, run QC, package sources and references.

Governance layer

Track licenses, source citations, model/version metadata, human approvals, test results, changes and published outputs.

3. Knowledge ingestion workflow

Step 1 — Discover

Start with a small, deliberate course collection. Store the course title, institution, author, URL, format, access date, license, reuse conditions and known limitations.

Step 2 — Extract

Use format-specific extractors for web pages, PDFs, captions and notebooks. Preserve section headings, equation text, page numbers and transcript timestamps where available. Route poor-quality OCR or broken equations to manual review.

Step 3 — Segment

Split content by meaningful units such as definition, theorem, derivation, example, experiment, misconception and exercise—not only by arbitrary token count.

Step 4 — Attach provenance

Every extracted claim should point to a source URL plus section/page/timestamp where possible. Store raw source text separately from AI-generated summaries.

Step 5 — Structure

Convert concepts and relations into validated records. Distinguish source statements, teacher notes, model-generated interpretations and unresolved disagreements.

Step 6 — Verify

Check equations and derivations computationally where feasible, compare independent sources, and route important or ambiguous claims to the teacher.

Step 7 — Index

Create keyword/full-text search first; add embeddings and a vector database when retrieval quality needs them. Add a graph for prerequisites and conceptual relations.

Step 8 — Refresh

Record source versions and re-check links or updated course materials periodically. Do not overwrite a reviewed record without a traceable revision.

4. Recommended staged technology choices

Prototype

Python, Git/GitHub, Manim Community Edition, SymPy, NumPy, Matplotlib, Jupyter/Colab, Markdown/YAML, and a simple folder-based source catalog.

Early knowledge base

SQLite or structured JSON/YAML for metadata and approved concept records; local full-text search; source files stored with clear licensing notes.

Growing knowledge base

PostgreSQL for structured records; a vector index for semantic retrieval if needed; a graph representation for prerequisite and concept relations. Choose components based on demonstrated need, not fashion.

Production

GitHub/Codespaces or a local development environment; pinned dependencies; render scripts; FFmpeg; audio editor; DaVinci Resolve or equivalent; repeatable export and QA checklist.

Later capabilities

Automated extraction queues, OCR quality checks, job scheduling, render workers, experiment tracking, formal proof tooling for selected mathematics, and a review dashboard.

5. Lecture compiler design

Input contract

Topic, audience, prerequisites, learning outcomes, target duration, source collection, visual style guide, language, assessment target and constraints.

Intermediate representation

A versioned machine-readable lecture document with claims, citations, equations, assumptions, narrative beats, scene IDs, visual objects, timing cues, narration, tests and approval status.

Compiler stages

Retrieve evidence → build concept map → propose teaching sequence → draft script → verify claims → storyboard → create scene specification → generate code → render preview → evaluate → teacher approval → assemble final.

Separation of concerns

Keep scientific model code separate from animation code; keep lesson meaning separate from rendering implementation; keep narration text separate from timestamps; keep source passages separate from AI summaries.

Failure behavior

If a citation is missing, an equation test fails, the schema is invalid, or a critical claim is disputed, stop the pipeline and flag the item. Do not silently fabricate a source or continue to publication.

6. Great thinkers and teaching methods library

Purpose

Represent documented ideas and teaching principles, not fictional replicas of famous people. Build original SaemPhysics lessons informed by conceptual clarity, mathematical structure, experimental reasoning, proof, symmetry, computation and historical context.

Record fields

Thinker or tradition; historical context; documented works; source references; core ideas; teaching principle inferred from the work; caveats; and example lesson applications.

Possible traditions

Feynman-inspired physical questioning; Euler's connections across mathematical structures; Faraday's experimental and field intuition; Noether's symmetry and conservation; Turing's computational models; Shannon's information-theoretic thinking; Newtonian and Hamiltonian mechanics; Maxwellian field theory.

Attribution safeguards

Do not invent quotations, attribute undocumented teaching methods, reproduce protected lecture scripts, or clone a living creator's distinctive expression. Cite the historical work and clearly label modern interpretation.

7. Quality and evaluation framework

Scientific quality

Correct equations, valid assumptions, dimensional consistency, correct signs and directions, sensible limiting cases, numerical checks, domain restrictions, source traceability.

Pedagogical quality

One clear central question, visible prerequisites, conceptual progression, meaningful visuals, explicit assumptions, appropriate cognitive load, and an assessment that tests the stated outcome.

Visual quality

Legible labels, consistent encoding, correct axes and units, smooth but meaningful transformations, no misleading physical implications, mobile readability and no clipping.

Engineering quality

Reproducible environment, pinned dependencies, testable model functions, successful preview render, manageable runtime, logged errors and versioned artifacts.

Release gate

A teacher approves the physics, narrative, preview and final export. Automation may flag and prepare; it should not publish critical science content without review.

8. 24-month roadmap

Months 0–2: animation foundation

Build the repository, environment, Manim primitives, render presets, review templates and 3–5 short scenes. Exit: repeatable scene from a clean checkout.

Months 2–4: mathematical engine

Add symbolic and numerical checks, graphing helpers, testable physical models and scene specifications. Exit: a model can be tested separately from its animation.

Months 3–6: knowledge ingestion

Catalog a small set of licensed OCW courses and open resources; extract and review concepts with source provenance. Exit: retrieved claims link back to source locations.

Months 5–9: lecture compiler MVP

Generate a sourced outline, storyboard, scene specification and starter Manim code for a narrow class of lessons. Exit: one full episode produced with human review.

Months 9–15: production automation

Add audio cues, caption workflow, reusable asset library, render testing and release packaging. Exit: a documented, repeatable production pipeline.

Months 15–24: cross-disciplinary expansion

Add calculus, linear algebra, probability, algorithms, data structures, numerical methods, mechanics, electromagnetism and selected advanced topics. Exit: domain-specific content uses shared infrastructure and dedicated validation.

9. First 30-day action plan

Week 1

Set up the repository and Manim; render one scene; document the environment and brand guide.

Week 2

Build reusable text, equation, vector, axes and graph components; create tests and examples.

Week 3

Produce a short electric-field or calculus episode with a verified derivation and storyboard.

Week 4

Record narration, edit, correct captions, publish if ready, and document production time and defects. Catalog only a small set of source materials as a pilot.

10. Starter data schema

Illustrative concept record

```
concept_id: calculus.derivative.rate_of_change
title: Derivative as instantaneous rate of change
domain: mathematics.calculus
prerequisites:
  - functions
  - slope_of_a_secant
claims:
  - claim_id: derivative.limit_definition
    statement: "Derivative is defined by a limit of difference quotients"
    source_refs: []
    verification_status: needs_review
visuals:
  - shrinking_secant_to_tangent
assessments:
  - predict_slope_from_graph
review_status: draft
```

Schema principle

This is illustrative YAML, not a complete validated schema. In production, define required fields, allowed values, versioning, and automated validation. Keep source claims, AI summaries and teacher-approved statements distinct.

11. Risks and safeguards

Copyright and licenses

OCW materials have specific reuse conditions; do not assume all materials can be downloaded, republished or used commercially. Track attribution and license metadata. Generate original explanations and animations rather than republishing source lectures.

Incorrect extraction

PDF OCR and equation extraction can corrupt symbols. Preserve the original source and flag low-confidence equations for manual review.

Hallucinated citations

Only cite sources that the retrieval system actually found. Validate URLs and section references. Do not allow an AI to invent a page number or quote.

Model agreement

Two models agreeing is not proof. Use independent calculations, trusted references, tests and teacher review.

Overengineering

Do not start with agents, a large vector database, a complex web application or automatic publishing. Build the smallest useful pipeline and expand after real use.

Security and privacy

Keep credentials out of source control and prompts. Review shell commands and package sources. Do not ingest private student data without an appropriate legal and privacy basis.

12. Definition of success

By the end of year one

The teacher can select a topic, retrieve source-grounded concepts, approve a derivation, generate a storyboard and scene code, render a preview, and assemble a complete video using a repeatable process.

Long-term measure

Measure correctness, student understanding, production time, revision rate, rendering reliability, audience retention and reuse—not simply the number of AI-generated videos.

Operating doctrine

Source-grounded knowledge; explicit assumptions; independent verification; one scene per teaching purpose; reusable components; visible human approval; and continuous improvement.